

Guion para video de sustentación — Actividad #9

Duración objetivo: 8-10 minutos (dentro del rango 5-10 min que pide la guía). Habla con naturalidad, no leas palabra por palabra: usa esto como guía de puntos, con más profundidad técnica y justificación de decisiones para que se note dominio del tema, no solo memorización.

0. Instrucciones técnicas: cómo grabar, paso a paso

A. Prepara las pestañas y ventanas ANTES de grabar

Abre esto con anticipación, en este orden, para no perder tiempo buscando durante la grabación:

1. **VS Code** (o el editor que uses) con la carpeta del proyecto abierta:
C:\Users\USUARIO\Desktop\SISTEMA-HELP-DESK. Ten listos en pestañas:
 - backend/src/models/Ticket.js
 - backend/src/controllers/ticketController.js
 - backend/src/routes/tickets.js
 - backend/src/server.js
 - frontend/js/config.js
 - frontend/js/api.js
 - frontend/js/app.js
2. **Navegador, pestaña 1:** <https://sistema-help-desk-green.vercel.app> (el frontend en producción).
3. **Navegador, pestaña 2:** <https://github.com/MaxVarela240/SISTEMA-HELP-DESK> (el repositorio).
4. **Navegador, pestaña 3:** <https://dashboard.render.com> → entra al servicio helpdesk-backend (para mostrar que está "Live").
5. **Navegador, pestaña 4:** <https://vercel.com/dashboard> → entra al proyecto sistema-help-desk (para mostrar el deployment).
6. Cierra cualquier notificación, chat o ventana que no quieras que se vea en pantalla.

B. Cómo grabar pantalla + tu rostro en Windows (gratis, sin instalar nada)

Opción recomendada — Xbox Game Bar (ya viene instalado en Windows 11):

7. Presiona **Windows + G** para abrir la Game Bar.
8. Conecta tu cámara web: en la Game Bar busca el widget de **Cámara** (ícono de cámara). Si no aparece, ábrelo desde el buscador de Windows escribiendo "Cámara" para confirmar que tu webcam funciona, luego vuelve a la Game Bar.
9. En el widget **Captura** (ícono de círculo rojo), asegúrate de que el micrófono esté activado (ícono de micrófono encendido).

10. Activa el widget de cámara para que tu rostro aparezca en una ventana flotante superpuesta — esto es lo que exige la guía ("mostrando su rostro").
11. Haz clic en **Iniciar grabación** (círculo rojo) o presiona **`Windows + Alt + R`**.
12. Sigue el guion de abajo (secciones 1 a 6).
13. Para terminar, presiona **`Windows + Alt + R`** de nuevo, o usa la barra de captura.
14. El video se guarda automáticamente en:
C:\Users\<tu_usuario>\Videos\Captures.

Si tu rostro no aparece superpuesto con Game Bar (a veces el widget de cámara no se combina bien con la grabación de pantalla completa), alternativa simple:

- Graba tu presentación inicial y de cierre (secciones 1 y 6 del guion) con la app **Cámara** de Windows, mostrando tu rostro hablando a cámara.
- Graba la demostración técnica (secciones 2 a 5) con Game Bar mostrando solo pantalla.
- Une ambos videos con el editor de **Clipchamp** (viene preinstalado en Windows 11: búscalo en el menú inicio) arrastrando los clips en orden y exportando como un solo archivo.

C. Cómo subir el video sin restricciones

YouTube (recomendado, más simple):

15. Ve a <https://studio.youtube.com> y haz clic en el ícono de **subir video** (cámara con un +) arriba a la derecha.
16. Arrastra tu archivo de video.
17. En "Detalles", pon un título como "Actividad 9 - Sustentación - Max Varela".
18. En **Visibilidad**, elige **"Público"** o **"No listado"** (No listado = solo quien tenga el link lo ve, cumple con "sin restricciones de privacidad" porque cualquiera con el enlace puede verlo sin iniciar sesión).
19. Copia el enlace que te da al finalizar.

Google Drive (alternativa):

20. Sube el archivo a tu Drive.
21. Clic derecho → **Compartir** → cambia el acceso a **"Cualquier persona con el enlace"** (no dejarlo en "Restringido").
22. Copia el enlace.

Pega ese enlace en el documento Varela_Max_Actividad9.docx, reemplazando el texto resaltado en amarillo.

Cómo leer cada sección de aquí en adelante

Cada parte del guion tiene dos bloques, siempre en el mismo orden:

- 🖱️ **QUÉ HACER** → son instrucciones para ti, el operador (dónde hacer clic, qué mostrar en pantalla). **Esto NO se dice en voz alta**, es solo tu guía visual mientras grabas.
- 🗣️ **QUÉ DECIR** → es el texto entre comillas que sí hablas frente a la cámara, con tus propias palabras (no hace falta memorizarlo exacto, solo la idea).

1. Introducción (40-50 seg)

🖱️ QUÉ HACER:

- Cámara/rostro visible en pantalla completa (sin compartir aún ninguna ventana de código o navegador).

🗣️ QUÉ DECIR:

"Hola, mi nombre es Max Varela, estudiante de Tecnologías de la Información en Línea, quinto semestre, paralelo A. Este es el video de sustentación de la Actividad número 9: el proyecto final de la asignatura Desarrollo de Sistemas Informáticos.

>

A lo largo del semestre construimos por partes un sistema de gestión de incidentes tipo Help Desk: primero el diseño y maquetado visual, luego la base de datos y la API REST en la Actividad 8, y ahora, en esta entrega final, el frontend que consume esa API, todo desplegado en servicios en la nube y funcionando en producción, sin depender de mi computadora local.

>

En los próximos minutos les voy a mostrar la arquitectura del sistema, voy a recorrer el código tanto del backend como del frontend, voy a hacer una demostración en vivo del sistema funcionando en internet, y voy a mostrarles dónde está desplegada cada parte."

2. Arquitectura general (1-1.5 min)

🖱️ QUÉ HACER:

- Puedes seguir con tu rostro en pantalla completa, o si tienes un diagrama simple dibujado (Frontend en Vercel → API REST en Render → MongoDB Atlas), muéstralo ahora. No es obligatorio, es opcional.

🗣️ QUÉ DECIR:

"El sistema tiene tres capas, cada una desplegada de forma independiente en la nube, comunicándose entre sí por internet.

>

Primero, el frontend: es una SPA, una Single Page Application, construida con HTML, CSS y JavaScript puro, sin frameworks pesados como Angular o React. La organicé en tres componentes bien separados: Dashboard, que muestra estadísticas; Formulario, para crear tickets nuevos; y Lista, para ver, actualizar y eliminar los existentes. Cada componente tiene su propio archivo JavaScript, lo que facilita mantener el código ordenado.

>

Segundo, el backend: es una API REST construida con Node.js y el framework Express, que expone operaciones CRUD — crear, leer, actualizar y eliminar — sobre el recurso 'ticket'. Sigue el patrón arquitectónico MVC, separando modelos, controladores y rutas.

>

Y tercero, la base de datos: MongoDB, alojada en MongoDB Atlas, un servicio cloud administrado, así que no depende de ningún servidor local ni de mi máquina.

>

La comunicación entre capas es así: el frontend le hace peticiones HTTP asíncronas a la API usando la función fetch de JavaScript; la API recibe esas peticiones con Express, las procesa, y usa Mongoose — que es un ODM, un mapeador de objetos a documentos — para leer y escribir en MongoDB Atlas. Todo esto viaja por internet en formato JSON."

2.1. Justificación de decisiones técnicas (1 min) — sección extra recomendada

QUÉ HACER:

- Puedes seguir hablando a cámara, sin necesidad de compartir pantalla todavía.

QUÉ DECIR:

"Quiero explicar brevemente por qué tomé estas decisiones técnicas.

>

Elegí Express sobre Node.js puro porque simplifica muchísimo el manejo de rutas, middlewares y respuestas HTTP, y es el estándar de facto en la industria para APIs REST en JavaScript.

>

Elegí MongoDB, una base de datos NoSQL orientada a documentos, porque el modelo de un ticket de soporte es relativamente flexible y no requiere relaciones complejas entre tablas — encaja natural en un documento JSON.

>

Para el frontend, en lugar de usar un framework como Angular, decidí construir la SPA con JavaScript puro para tener control total sobre cada parte del código y demostrar que entiendo los fundamentos — el manejo del DOM, las peticiones fetch, el manejo de estado — sin depender de abstracciones de un framework.

>

Y para el despliegue, elegí específicamente MongoDB Atlas, Render y Vercel porque los tres ofrecen niveles gratuitos suficientes para un proyecto académico, se integran directamente con GitHub, y son plataformas ampliamente usadas en la industria actualmente."

3. Recorrido del código (3-4 min)

QUÉ HACER:

- Cambia a la ventana de **VS Code**.
- Abre y muestra brevemente, en este orden: `backend/src/models/Ticket.js` → `backend/src/controllers/ticketController.js` → `backend/src/routes/tickets.js` → `backend/src/config/db.js` → `backend/src/server.js`.
- Luego cambia a: `frontend/js/config.js` → `frontend/js/api.js` → `frontend/js/dashboard.js` → `frontend/js/formulario.js` → `frontend/js/lista.js` → `frontend/js/app.js`.
- No necesitas leer el código línea por línea, solo señala con el cursor las partes que menciones al hablar. Ve despacio, dale 3-4 segundos a cada archivo para que se alcance a leer en el video.

QUÉ DECIR (al abrir `Ticket.js`, el modelo):

"Empiezo por el modelo, en `models/Ticket.js`. Aquí defino con Mongoose el esquema del ticket: título, descripción y solicitante son campos de texto obligatorios; categoría es un campo restringido a tres valores posibles — Red, Hardware o Software — usando un enum, lo que evita datos inconsistentes; prioridad y estado también usan enums, con valores por defecto 'Media' y 'Abierto' respectivamente. Además, activé la opción `timestamps`, que agrega automáticamente la fecha de creación y actualización a cada documento."

QUÉ DECIR (al abrir `ticketController.js`):

"En el controlador está la lógica de negocio de cada operación. La función `listarTickets` hace una consulta `find` y ordena por fecha de creación descendente, para ver los más recientes primero. `crearTicket` recibe los datos del cuerpo de la petición y los guarda con `create`. `actualizarTicket` usa `findByIdAndUpdate` con la opción `runValidators` activada, para que se respeten los enums del esquema incluso

al actualizar. Y `eliminarTicket` usa `findByIdAndDelete`. Cada función maneja sus errores con `try-catch` y devuelve el código de estado HTTP apropiado — 404 si no se encuentra el ticket, 400 si los datos son inválidos, 500 si hay un error del servidor."

QUÉ DECIR (al abrir ``tickets.js``, las rutas):

"Las rutas son el punto de entrada HTTP: mapean cada verbo y cada URL — GET, POST, PUT, DELETE sobre `/api/tickets` — a su función correspondiente en el controlador. Esto separa claramente 'qué endpoint existe' de 'cómo se procesa cada uno', que es justo la idea del patrón MVC."

QUÉ DECIR (al abrir ``server.js``):

"Y en `server.js` arranco la aplicación: configuro Express, habilito CORS para que el frontend, que vive en otro dominio, pueda hacer peticiones sin ser bloqueado por el navegador, habilito el parseo de JSON en el cuerpo de las peticiones, monto las rutas de tickets bajo `/api/tickets`, y antes de levantar el servidor me conecto a MongoDB usando la variable de entorno `MONGODB_URI` — así nunca dejo credenciales escritas directamente en el código."

QUÉ DECIR (mientras muestras el frontend):

"Del lado del frontend, `config.js` centraliza la URL base de la API en una sola constante, así que si algún día cambio de proveedor de hosting, solo edito un archivo. `api.js` encapsula todas las llamadas fetch — listar, crear, actualizar, eliminar — en un solo objeto `TicketAPI`, para no repetir código en cada vista. Cada vista, Dashboard, Formulario y Lista, tiene su propio archivo con un método `render` que construye su HTML dinámicamente según los datos que recibe de la API. Y `app.js` hace de router simple: escucha los clics en el menú de navegación y muestra u oculta cada sección."

4. Demostración en vivo (2 min)

QUÉ HACER:

23. Cambia a la **pestaña 1** del navegador (<https://sistema-help-desk-green.vercel.app>). Debe abrir en Dashboard.
24. Clic en **"Nuevo Ticket"** (menú azul, arriba).
25. Llena el formulario mientras hablas:

- Título: Impresora sin conexión en oficina 2
- Solicitante: tu nombre
- Descripción: cualquier frase corta
- Categoría: "Hardware"
- Prioridad: "Alta"

26. Clic en **"Crear ticket"** (te lleva automáticamente a la Lista).
27. Señala con el mouse la fila del ticket recién creado.
28. Clic en el desplegable de **Estado** de esa fila → cámbialo a **"En Progreso"**. Espera el mensaje de confirmación abajo a la derecha.
29. Clic en **"Dashboard"** (arriba) para volver. Señala que las tarjetas cambiaron de número.
30. *(Opcional, si te sobra tiempo)* Vuelve a Lista, clic en **"Eliminar"** en el ticket de prueba, confirma en el diálogo.

QUÉ DECIR (al llegar al Dashboard, paso 1):

"Estamos viendo el sistema en su URL pública real, no en localhost. Aquí está el Dashboard, con el resumen en tiempo real: total de tickets, abiertos, en progreso y cerrados. Estos números salen de una consulta a la API que corre en el momento en que carga la página."

QUÉ DECIR (al crear el ticket, pasos 2-4):

"Voy a crear un nuevo ticket para simular un caso real de soporte: título 'impresora sin conexión en oficina 2', el nombre del solicitante, una breve descripción del problema, categoría hardware, y prioridad alta porque afecta el trabajo de una oficina completa. Al enviar el formulario, el navegador hace una petición POST a la API con estos datos en formato JSON."

QUÉ DECIR (al verlo en la lista, paso 5):

"El sistema me redirige automáticamente a la Lista de tickets, y ahí está mi ticket recién creado, ya persistido en MongoDB Atlas. Si en este momento recargara la página, o incluso si la abriera desde otro dispositivo, el ticket seguiría ahí, porque no vive en la memoria del navegador sino en la base de datos en la nube."

QUÉ DECIR (al cambiar el estado, paso 6):

"Ahora voy a simular que un técnico toma este ticket: cambio su estado de Abierto a En Progreso desde este mismo desplegable, sin recargar la página. Esto dispara una petición PUT a la API, que actualiza el documento en MongoDB, y el sistema me confirma el cambio con este mensaje."

QUÉ DECIR (al volver al Dashboard, paso 7):

"Y si vuelvo al Dashboard, los contadores ya reflejan el cambio: bajó uno en 'Abiertos' y subió uno en 'En Progreso'. Todo esto está pasando en tiempo real, entre un frontend en Vercel, una API en Render y una base de datos en MongoDB Atlas — tres servicios distintos, en la nube, conversando entre sí por internet, sin que nada dependa de mi computadora."

5. Repositorios y despliegue (1 min)

QUÉ HACER:

31. Cambia a la **pestaña 2** (GitHub): muestra la lista de archivos y, si quieres, entra a "commits" para mostrar el historial.
32. Cambia a la **pestaña 3** (Render): muestra que el servicio helpdesk-backend dice **"Live"** en verde.
33. Cambia a la **pestaña 4** (Vercel): muestra que el proyecto tiene un deployment marcado como **"Ready"**.

QUÉ DECIR (puedes hablar mientras cambias entre las 3 pestañas):

"El código está organizado en un único repositorio de GitHub, con dos carpetas principales: backend y frontend. Seguí un flujo de ramas tipo Git Flow: cada funcionalidad nueva la desarrollé en una rama feature, por ejemplo feature/backend-api o feature/frontend-spa; esas ramas se integran a develop, que es la rama de trabajo activa; y develop se fusiona a main, que es la rama estable, la que efectivamente se despliega en producción. Esto se refleja en el historial de commits, con mensajes descriptivos siguiendo la convención de commits convencionales.

>

El backend está desplegado en Render como un Web Service gratuito, apuntando directamente a este repositorio y a la carpeta backend; configuré ahí la variable de entorno MONGODB_URI para que se conecte a mi cluster de MongoDB Atlas. Pueden ver que el estado del servicio dice 'Live'.

>

Y el frontend está desplegado en Vercel como un sitio estático, apuntando a la carpeta frontend del mismo repositorio; cada vez que hago push a la rama main, Vercel vuelve a desplegar automáticamente la última versión."

5.1. Retos, pruebas y aprendizajes (1 min) — sección extra recomendada

QUÉ HACER:

- Puedes volver a hablar a cámara, o mostrar de nuevo brevemente la pestaña de Render/logs si quieres reforzar visualmente.

QUÉ DECIR:

"Durante el desarrollo enfrenté algunos retos típicos de un despliegue full stack real. Por ejemplo, tuve que asegurarme de habilitar CORS correctamente en el backend, porque el navegador bloquea por defecto las peticiones entre dominios distintos — el frontend en Vercel y la API en Render son dominios diferentes. También tuve que

configurar la lista de acceso de red en MongoDB Atlas para permitir conexiones desde Render, ya que por defecto solo permite la IP desde donde se creó el cluster.

>

Para probar la API antes de conectarla al frontend, primero la validé de forma local, verificando cada endpoint con datos de prueba, y luego repetí esas mismas pruebas ya contra la URL pública en Render, para confirmar que el comportamiento fuera idéntico en producción.

>

Este proceso me permitió entender de forma práctica conceptos que solo había visto en teoría: la diferencia entre un entorno de desarrollo y uno de producción, la importancia de las variables de entorno para no exponer credenciales, y cómo coordinar el despliegue de múltiples servicios independientes que deben funcionar como un solo sistema."

6. Cierre (30-40 seg)

QUÉ HACER:

- Vuelve a mostrar tu rostro en pantalla completa (cierra o minimiza las ventanas de código/navegador).

QUÉ DECIR:

"Con esto concluyo la sustentación de la Actividad número 9. A lo largo del video mostré la arquitectura del sistema, el código del backend y del frontend, una demostración en vivo del sistema funcionando en internet, y la infraestructura donde está desplegado.

>

El resultado es un sistema completo — frontend, backend y base de datos — funcionando de manera integrada, desplegado en la nube y operativo sin depender de servidores locales, cumpliendo con todos los requisitos de la actividad y cerrando el proyecto semestral de Desarrollo de Sistemas Informáticos.

>

Muchas gracias por su atención."

URLs del proyecto desplegado

- Repositorio GitHub: <https://github.com/MaxVarela240/SISTEMA-HELP-DESK>
- Backend (API REST) en Render: <https://helpdesk-backend-oy9a.onrender.com/api/tickets>
- Frontend (SPA) en Vercel: <https://sistema-help-desk-green.vercel.app>

- Base de datos: MongoDB Atlas (cluster Cluster0)

Checklist antes de grabar

- ☐ Backend desplegado y respondiendo en su URL pública (no localhost).
- ☐ Frontend desplegado apuntando a la URL del backend (revisar frontend/js/config.js).
- ☐ Al menos 1-2 tickets de prueba creados para que el Dashboard no se vea vacío.
- ☐ Mostrar tu rostro durante el video (requisito obligatorio).
- ☐ Subir el video sin restricciones de privacidad (público o "cualquiera con el enlace").